



# Dijkstra and Bellman–Ford

## 1 Shortest Paths

Recall that a weighted graph  $G$  is a graph equipped with a weight function  $w : E \rightarrow \mathbb{R}$ , where the weight of a path is defined as the sum of the weights of the edges along the path. The shortest path between two vertices is a path that connects them and is also minimal in its weight. One native application of this formulation is to model routes between destinations, where edges correspond to roads and vertices to mid-stations.

Two main algorithms are used to solve the shortest path problem. The first is Dijkstra, which is required only when the weight function is **non-negative** and runs in time  $\Theta(|E| \log |V|)$ <sup>1</sup>. The second is Bellman-Ford, which handles **negative** weights, yet it does so at a higher running time cost of  $\Theta(|E| \cdot |V|)$ .

Initially the distances from the source are initialized to  $\infty$  and along the running they are decreased closer to their real value by application of relaxations.

**Claim 1.** *Given a weighted (can be directed) graph  $G$  with weights  $w : E \rightarrow \mathbb{R}$ , let  $v_0, v_1, \dots, v_k$  be a shortest path from  $v_0$  to  $v_k$  then for any  $0 \leq i < j \leq k$  the path  $v_i, v_{i+1}, \dots, v_{j-1}, v_j$  is a shortest path from  $v_i$  to  $v_j$ .*

*Proof.* Assume by contradiction that there is a path  $v_i, u_{i+1}, \dots, u_{j-1}, v_j$  that has a lower weight than  $v_i, v_{i+1}, \dots, v_{j-1}, v_j$  then:

$$\begin{aligned} w(v_0, v_1, \dots, v_k) &= w(v_0, v_1, \dots, v_i) + w(v_i, v_{i+1}, \dots, v_j) + w(v_j, v_{j+1}, \dots, v_k) \\ &> w(v_0, v_1, \dots, v_i) + w(v_i, u_{i+1}, \dots, u_{j-1}, v_j) + w(v_j, v_{j+1}, \dots, v_k) \\ &= w(v_0, v_1, \dots, v_i, u_{i+1}, \dots, u_{j-1}, v_j, \dots, v_k) \end{aligned}$$

□

### Relaxation.

```
1 if  $d[v] > d[u] + w(u, v)$  then
2   |  $d[v] \leftarrow d[u] + w(u, v)$ 
3   |  $\pi[v] \leftarrow u$ 
4 end
```

**Algorithm 1:** Relaxation.

<sup>1</sup>Can be improved to  $|E| + |V| \log |V|$ , but that is beyond the scope of the course.

**Shortest-paths Properties.** Let  $G$  be (can be directed) a graph, For any  $v, u \in E$  denote by  $\delta(v, u)$  the length of a shortest path from  $v$  to  $u$ . For a source vertex  $s \in V$  and any vertex  $v \in V$ , associate a register  $d[v]$  initialize as:

$$d[v] = \begin{cases} \infty & v \neq s \\ 0 & v = s \end{cases}$$

Then the following properties hold:

- **Triangle inequality.** For all edges  $(u, v) \in E$  we have:

$$\delta(s, v) \leq \delta(s, u) + w(u, v)$$

*Proof.* By definition,  $\delta(s, v)$  is the weight of the minimal path from  $s$  to  $v$ , thus it lower or equal than the weight of any other path from  $s$  to  $v$ , Particularly the weight of path obtained by chaining a shortest from  $s$  to  $u$  with the edge from  $u$  to  $v$ . The weight of such path is  $\delta(s, u) + w(u, v)$  and we get the desired inequality.  $\square$

- **Upper-bound property.** After application of any sequence relaxation steps we have  $d[v] \geq \delta(s, v)$ .

*Proof.* By induction on the  $i$ th relaxation step. We present only the induction step. If  $d[v]$  was changed after the step, then it was setted to be  $d[u] + w(u, v)$  for some  $u \in V$  such  $(u, v) \in E$ . However:

$$\begin{aligned} d[u] + w(u, v) &\geq \delta(s, u) + w(u, v) \\ &\geq \delta(s, v) \end{aligned}$$

Where the first inequality uses the induction assumption for the  $i - 1$  previous relaxation steps, and the second is follow by the triangle inequality.  $\square$

- **Convergence property.** Suppose that a shortest path from  $s$  to  $v$  crosses through  $(u, v) \in E$  and suppose that  $d[u] = \delta(s, u)$  then after application of relaxation  $(u, v)$  we have  $d[v] = \delta(s, v)$ .

*Proof.* By claim 1, any sub path of a shortest path is also a shortest path between its two ends. Therefore  $\delta(s, v) = \delta(s, u) + w(u, v)$ , combining the given that  $d[u]$  equals to  $\delta(s, u)$  we get the desired equivalence.  $\square$

- **Path-relaxation property.** Let  $s = v_0, v_1, v_2, \dots, v_k$  be a shortest path from  $s$  to  $v_k$ . After an application of a sequence relaxation steps that includes, in order,  $(v_0, v_1), (v_1, v_2), \dots, (v_{k-1}, v_k)$  we have that  $d[v_k] = \delta(s, v_k)$ .

*Proof.* By induction on the  $i$ th (relevant) relaxation step, namely:  $(v_{i-1}, v_i)$ . We present only the induction step. By the induction assumption after the  $i - 1$ th relevant step, we have that  $d[v_{i-1}] = \delta(s, v_{i-1})$ . In addition, the **Upper-bound property** guarantees that following relaxations steps can't

decrease (change) that value. Therefore, at the beginning of the  $i$ th relevant relaxation step,  $d[v_{i-1}]$  is still equals to  $\delta(s, v_{i-1})$  and from the **Convergence property** the application of the  $(v_{i-1}, v_i)$  relaxation sets  $d[v_i]$  to be  $\delta(s, v_i)$ .

□

*Remark 1.* Corman defines a few more properties and uses them to prove that the returned predecessor forest  $\pi$  can be used to restore the paths. In this recitation, we will be satisfied with proving that the algorithms compute the lengths of the shortest paths. Yet, the generalization is not hard.

## 2 Dijkstra

Dijkstra can be thought of as BFS where the FIFO queue is replaced by a min-heap. Each time we extract the closest unvisited vertex, we scan its edges and check for each of its neighbors if getting to it through their edge defines a shorter path than what is currently known. If so, we update the heap. Algorithm 2 gives an implementation.

**What's wrong with negative weights?** Consider the following graph,

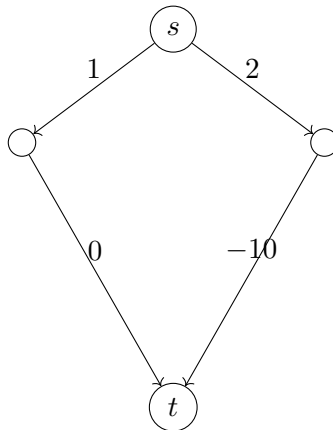


Figure 5.1: Example of a binary tree with 12 vertices. In blue, one of the longest paths from the root to one of the leaves, which is of length 4.

**Claim 2** (Dijkstra Correctness). *At the time that  $u$  is being extracted out of  $Q$ , we have  $d[u] = \delta(s, u)$ .*

*Proof.* Assume for the sake of contradiction that the claim is false, and let  $u$  be the first vertex that at the time its being extracted, the value of  $d[u]$  is strictly greater than  $\delta(s, u)$ . If  $u = s$  the contradiction is immediate. By the **Upper-bound property**, if  $u$  is not reachable from  $s$  then  $d[u] > \infty$  for the whole running, and the again the contradiction follows directly. Hence it's left to deal the case where  $u$  is reachable from  $v$ . Let  $x$  and  $y$  be a two adjacent vertices on a shortest path  $s \rightsquigarrow u$ , such  $x$  has extracted out from  $Q$  before  $u$  and  $y$  not. Notice that  $x$  can be equals to  $s$  and  $y$  can be equals to  $u$ .

```

1 for  $v \in V$  do
2    $d[v] \leftarrow \infty$ 
3    $\pi[v] \leftarrow \text{null}$ 
4 end
5  $d[s] \leftarrow 0$ 
6  $Q \leftarrow \text{Heap}(d[v_1], d[v_2], \dots, d[v_n])$ 
7 while  $Q$  is not empty do
8    $u \leftarrow \text{extract-min}(Q)$ 
9   for  $(u, v) \in E$  do
10    if  $d[v] > d[u] + w(u, v)$  then
11       $d[v] \leftarrow d[u] + w(u, v)$ 
12       $\text{Update}(Q, d[v])$ 
13       $\pi[v] \leftarrow u$ 
14    end
15  end
16 end

```

**Algorithm 2:** Dijkstra.

From the assumption that  $u$  is the first vertex to be extracted out that violates the claim, we have that  $d[x] = \delta(s, x)$ , and by the **Convergence property** also  $d[y] = \delta(s, y)$ . Yet by the correctness of min-heap extraction, we have that  $d[u] \leq d[y]$ . Combing that  $\delta(s, y) \leq \delta(s, u)$  ( $\star$ ) we have:

$$\begin{aligned} \delta(s, y) &\leq \delta(s, u) \leq d[u] \leq d[y] = \delta(s, y) \\ \Rightarrow \delta(s, u) &= d[u] \end{aligned}$$

In contradiction to  $u$  be the first vertex to have a strictly greater  $d[u]$  than  $\delta(s, u)$  at the extraction time.  $\square$

**Exercise 1.** Where is the exactly points in which the proof fails for negative weight?

*Solution 1.* At ( $\star$ ), the inequality  $\delta(s, y) \leq \delta(s, v)$  is correct only if we assume that  $w \geq 0$ . In that case by **Triangle inequality** we have that for adjacent vertices on a shortest path  $x, y$  we have that:

$$\delta(s, x) = \delta(s, y) + w(y, x) \leq \delta(s, y)$$

On the other hand, without the assumption, one can think about the graph that consists of a single negative edge.

### 3 Bellman-Ford

Consider the graph with 4 vertices, denoted by  $V = \{1, 2, 3, 4\}$ . Assume that the shortest path from 1 to 4 is 1, 3, 2, 4, and consider the application of the following relaxation steps:

(1, 2), (1, 3), (1, 4), (2, 1), (2, 3), (2, 4), (3, 1), (3, 2), (3, 4), (4, 1), (4, 2), (4, 3)  
 (1, 2), (1, 3), (1, 4), (2, 1), (2, 3), (2, 4), (3, 1), (3, 2), (3, 4), (4, 1), (4, 2), (4, 3)  
 (1, 2), (1, 3), (1, 4), (2, 1), (2, 3), (2, 4), (3, 1), (3, 2), (3, 4), (4, 1), (4, 2), (4, 3)

Each row contains relaxations for all edges. Thus, in terms of the **Path-relaxation property**, it ensures that by the end of the  $i$ th row, the  $i$ th relevant relaxation will be called. That idea gives an algorithm that computes the shortest paths for a graph with negative weights.

```

1 for  $v \in V$  do
2    $d[v] \leftarrow \infty$ 
3    $\pi[v] \leftarrow \text{null}$ 
4 end
5  $d[s] \leftarrow 0$ 
6 for  $i \in [n - 1]$  do
7   for  $(u, v) \in E$  do
8     if  $d[v] > d[u] + w(u, v)$  then
9        $d[v] \leftarrow d[u] + w(u, v)$ 
10       $\pi[v] \leftarrow u$ 
11    end
12  end
13 end
14 for  $(u, v) \in E$  do
15   if  $d[v] > d[u] + w(u, v)$  then
16     return Negative Cycle
17   end
18 end

```

**Algorithm 3:** Bellman–Ford.

Showing that Bellman-ford detects negative cycle is left as exercise.

**Detecting a Negative Cycle.** Suppose that  $v_1, v_1, \dots, v_k = v_1$  is a negative cycle. This means that:

$$\sum_{i=2}^k w(v_{i-1}, v_i) < 0$$

Assume that the algorithm doesn't detect the cycle, namely the condition at Line 14 is not satisfied for any of the edges. Then:

$$\begin{aligned}
& \forall v_i \ d[v] < d[u] + w(u, v) \\
\Rightarrow & \sum_{i=2}^k d[v_i] < \sum_{i=2}^k d[v_{i-1}] + \sum_{i=2}^k w(v_{i-1}, v_i) \\
& = \sum_{i=2}^k d[v_i] + \sum_{i=2}^k w(v_{i-1}, v_i) \\
& 0 \leq \sum_{i=2}^k w(v_{i-1}, v_i)
\end{aligned}$$

**Correctness.** at the  $i$  iteration, we apply the relaxtion  $(v_{i-1}, v_i)$  after we apply it over  $(v_1, v_2), (v_2, v_3), \dots, (v_{i-2}, v_{i-1})$